

Guide : pull requests, mise en production et déploiement

Deux dépôts font vivre le site de l'agence OPTINOV :

Dépôt	Rôle	Hébergement	Adresse
christ544/optinov-tableau	le tableau de bord (Payload CMS)	Render	https://optinov-dashboard.onrender.com/admin
christ544/Optinov-agence	le site vitrine (Next.js, statique)	Cloudflare	https://optinov-agence.christkangah14.workers.dev

Tu es propriétaire des deux. Paul-Aymard propose les changements par **pull request** (PR) ; c'est toi qui relis et qui fusionnes. Ce guide explique comment relire, tester, fusionner, proposer toi-même un changement, et comment déployer sur PlanetHoster le jour où l'hébergement change.

1. Le principe

- Le travail est découpé en **étapes**. Chaque étape vit sur sa propre branche et donne **une PR vers main**.
- Les branches du tableau de bord sont **empilées** : l'étape 2 contient l'étape 1, l'étape 3 contient les étapes 1 et 2, etc. Il faut donc **fusionner dans l'ordre**.
- Tant que la PR #1 n'est pas fusionnée, l'onglet « Files changed » de la PR #2 montre aussi les fichiers de l'étape 1. C'est normal : il se réduit tout seul dès que #1 est fusionnée. Pour relire, ne regarde que ce que la description de la PR annonce.

PR	Dépôt	Branche	Contenu
#1	tableau de bord	etape-1-environnement-local	Cloudinary facultatif, images sur disque en local
#2	tableau de bord	etape-2-nettoyage	retrait du gabarit Cloudflare, ESLint réparé, README en français
#3	tableau de bord	etape-3-habillage-optinov	charte OPTINOV dans l'admin, accueil sur mesure, logo
#4	tableau de bord	etape-4-securite	rôles administrateur / éditeur, CORS et CSRF, uploads, GraphQL coupé ; contient une migration
#5	tableau de bord	etape-5-contenus	paramètres du site, réalisations, témoignages, équipe, FAQ ; contient deux migrations
site 1	site vitrine	dashboard-payload	le site lit le tableau de bord, seule source de vérité ; retrait des anciens outils d'édition
#6	tableau de bord	etape-6-demandes	rubrique Demandes, alertes e-mail par le Gmail de l'agence ; contient une migration
site 2	site vitrine	formulaire-vers-dashboard	les formulaires envoient les demandes au tableau de bord

Ordre à respecter : #1, #2, #3, #4, #5, puis la PR « site 1 » **après** que Render a redéployé le tableau de bord (section 5) ; ensuite #6, puis « site 2 » après le redéploiement suivant. Dans l'autre sens, la construction du site échoue exprès, parce que les nouvelles rubriques n'existent pas encore dans l'API.

Après la fusion de #6, renseigner dans Render trois variables : `SMTP_USER` (l'adresse Gmail de l'agence), `SMTP_PASS` (un mot de passe d'application Google, créé dans le compte Google > Sécurité > Validation en deux étapes > Mots de passe des applications) et `ALERTE_EMAIL` (l'adresse qui reçoit les alertes). Sans elles, les demandes sont enregistrées mais personne n'est prévenu.

2. Relire une PR (sur GitHub)

1. Ouvre la PR, lis la **description** dans l'onglet « Conversation » : elle dit ce qui change et ce qui a été vérifié.
2. Onglet « **Files changed** » : parcours les fichiers. Pour poser une question ou demander une correction, clique sur le `+` en face de la ligne et écris un commentaire.
3. Bouton « **Review changes** » (en haut à droite) :
 - **Approve** : tout est bon ;
 - **Request changes** : tu attends une correction avant de fusionner ;
 - **Comment** : simple remarque, sans bloquer.

Il n'y a pas d'intégration continue sur ces dépôts : GitHub ne lance aucun test à ta place. Le test, c'est toi qui le fais en local (section 3).

3. Tester en local avant de fusionner

Le tableau de bord

Prérequis : le projet installé selon son README (Node 22, PostgreSQL, fichier `.env`).

```
git fetch origin
git checkout etape-4-securite      # la branche de la PR à tester
npm install                       # les dépendances changent selon les étapes
npm run migrate                   # applique les migrations s'il y en a de nouvelles
npm run dev                       # puis http://localhost:3000/admin
```

Vérifications rapides, dans un second terminal :

```
npm run lint
npm run test:int
npm run test:e2e                  # la première fois : npx playwright install chromium
```

Le premier chargement de l'admin prend environ une minute : ce n'est pas un bug. Pour voir le tableau de bord rempli, `npm run seed` charge un jeu de démonstration (uniquement sur une base locale). Quand tu as fini : `git checkout main`.

Le site

Le site lit le tableau de bord au moment de sa construction. Pour le tester avec ton tableau de bord local lancé sur le port 3000 :

```
git fetch origin
git checkout dashboard-payload
npm install
DASHBOARD_URL=http://localhost:3000 npm run sync # récupère le contenu
npm run dev # puis http://localhost:3000 ... ou
3001 si le port est pris
```

Pour une construction complète, comme sur Cloudflare :

```
DASHBOARD_URL=http://localhost:3000 npm run build # produit le dossier out/
```

4. Fusionner

1. Fusionne **dans l'ordre des numéros** (#1 avant #2, #2 avant #3, etc.).
2. Clique sur la flèche du bouton vert et choisis « **Create a merge commit** ». Pas « **Squash and merge** », pas « **Rebase and merge** » : ces deux options réécrivent les commits, et GitHub croirait alors que la PR suivante contient encore l'étape précédente (diff gonflé, faux conflits).
3. « Delete branch » après la fusion : possible, sans risque, la branche suivante contient déjà ces commits.
4. Ouvre la PR suivante et vérifie que « Files changed » ne montre plus que ses propres fichiers. Si oui, tu peux la relire et la fusionner à son tour.
5. La PR du site se fusionne **en dernier**, une fois le tableau de bord redéployé.

5. Ce qui se passe après une fusion

Tableau de bord (Render)

- **Render redéploie main automatiquement** (`autoDeploy: true` dans `render.yaml`). Une fusion dans `main` = une mise en production. Fusionne donc uniquement ce que tu as relu et testé.
- Les **migrations** de base s'appliquent toutes seules au démarrage du service. Celle de l'étape 4 déclare administrateurs tous les comptes existants ; celles de l'étape 5 créent les nouvelles rubriques et y installent les vingt questions de la FAQ et les coordonnées du site.
- Surveille le déploiement dans Render (onglet « Events » puis « Logs »). Si le build échoue, le service précédent reste en ligne : rien n'est cassé, préviens Paul-Aymard avec le message d'erreur.
- Le service gratuit s'endort après quinze minutes sans visite et met jusqu'à une minute à se réveiller. Ce n'est pas une panne.

Site (Cloudflare)

- Après la fusion de la PR du site, lance une construction dans Cloudflare (Workers & Pages, projet `optinov-agence`, « Retry deployment » ou un nouveau déploiement), ou pousse un commit.

- Ensuite, **chaque enregistrement dans le tableau de bord reconstruit le site** tout seul, deux minutes après le dernier enregistrement, grâce au Deploy Hook renseigné dans Render (`SITE_DEPLOY_HOOK`). La modification est en ligne quelques minutes plus tard.
- Si le tableau de bord est injoignable pendant la construction, celle-ci échoue volontairement : la version précédente du site reste en ligne.

Sur ton PC, après une fusion : `git checkout main && git pull` .

6. Proposer toi-même une modification

La même règle vaut pour tout le monde : **jamais de commit directement sur `main`** , toujours une branche et une PR. Exemple sur le tableau de bord :

```
git checkout main && git pull          # partir de la version en ligne
git checkout -b correction-libelle-faq  # une branche au nom parlant
# ... modifier les fichiers ...
git add -A
git commit -m "FAQ : corrige le libellé du thème Support"
git push -u origin correction-libelle-faq # envoie la branche sur GitHub
gh pr create --base main --fill          # ouvre la PR (ou bouton « Compare & pull
request » sur GitHub)
```

- **Compléter une PR déjà ouverte** : fais tes commits sur **la même branche** et `git push` . La PR se met à jour toute seule.
- **Une PR par sujet** : une correction, une branche. Deux sujets mélangés rendent la relecture et le retour en arrière difficiles.
- **Avant de pousser** : `npm run lint` et `npm run test:int` doivent passer.
- **Migrations** : si tu modifies une collection, `npm run migrate:create` puis commite le fichier de migration avec ton code. Sans lui, la production démarre sur un schéma incomplet.

7. Demander une correction

Commente sur la ligne concernée ou choisis « Request changes ». L'auteur pousse un commit supplémentaire sur **la même branche** : la PR se met à jour toute seule, tu n'as rien à rouvrir. Relis le nouveau commit, puis approuve et fusionne.

8. En cas de conflit

Si GitHub affiche « This branch has conflicts that must be resolved », **ne résous pas le conflit dans l'éditeur GitHub** : avec des branches empilées, cela casse les PR suivantes. Préviens l'auteur, il rebase la branche et repousse ; le conflit disparaît.

9. Déployer sur PlanetHoster (NOC)

Aujourd'hui le tableau de bord tourne sur Render (gratuit, mais il s'endort) et le site sur Cloudflare. Si un jour le tableau de bord doit rejoindre l'hébergement PlanetHoster de l'agence, comme le site immobilier,

voici la marche à suivre. Le site vitrine, lui, peut rester sur Cloudflare : il est statique.

Ce qu'on crée dans le panneau N0C

Élément	Valeur
Sous-domaine	<code>admin.<domaine de l'agence></code>
Application Node.js	version 22, mode production, répertoire de l'application = le clone du dépôt
Base PostgreSQL	une base et un utilisateur dédiés (menu « Bases de données »)
Dossier des images	<code>~/media-optinov-agence</code> , à côté de l'application, jamais dedans (sinon il disparaît à chaque déploiement)

Les six règles de N0C

Elles ne sont écrites nulle part chez l'hébergeur, et chacune coûte une demi-journée quand on ne les connaît pas.

1. **Activer l'environnement Node avant tout** `npm` (la commande d'activation est affichée dans le panneau, en haut de l'application). Sans cela, `npm` installe avec le mauvais Node et l'application refuse de démarrer.
2. **Installer avec les dépendances de développement** : `NODE_ENV=development npm ci --include=dev`. Sans elles, TypeScript manque et le build échoue.
3. **Brider les ressources au build**, sinon le processus est tué (`os error 11`).
4. `node_modules` **est un lien symbolique à l'exécution, un vrai dossier au build**. `npm ci` remplace le lien par un dossier : on construit, **puis** on repose le lien. Jamais l'inverse.
5. **Le schéma se crée par migrations** (`npm run migrate`), jamais par `push` .
6. **Passenger garde le processus en mémoire** : après tout changement, `touch tmp/restart.txt` dans le dossier de l'application, ou ↻ dans le panneau.

Premier déploiement, en SSH sur le serveur

```
# 1. Le code
git clone https://github.com/christ544/optinov-tableau.git ~/optinov-admin
cd ~/optinov-admin

# 2. Les variables (voir la liste ci-dessous)
cp .env.example .env && nano .env

# 3. L'environnement Node de l'application (commande donnée par le panneau)
source ~/nodeenv/optinov-admin/22/bin/activate

# 4. Les dépendances, puis le schéma
NODE_ENV=development npm ci --include=dev
npm run migrate

# 5. La construction, ressources bridées, avec Webpack
NODE_OPTIONS="--no-deprecation --max-old-space-size=2048" \
NEXT_CPUS=1 RAYON_NUM_THREADS=1 TOKIO_WORKER_THREADS=1 UV_THREADPOOL_SIZE=1 \
  npx next build --webpack

# 6. Reposer le lien node_modules (règle 4)
mv node_modules ~/nodeenv/optinov-admin/22/lib/node_modules
ln -s ~/nodeenv/optinov-admin/22/lib/node_modules node_modules

# 7. Démarrer, puis vérifier
mkdir -p tmp && touch tmp/restart.txt
curl -s -o /dev/null -w "%{http_code}\n" https://admin.<domaine>/admin # attendu : 200
```

Variables d'environnement à renseigner

Variable	Valeur
DATABASE_URI	la chaîne de connexion de la base PostgreSQL NOC
PAYLOAD_SECRET	une chaîne aléatoire : <code>openssl rand -hex 32</code>
PAYLOAD_PUBLIC_SERVER_URL	<code>https://admin.<domaine></code> : exactement l'adresse par laquelle on ouvre l'admin, sinon tout enregistrement est refusé
FRONTEND_URL	l'adresse du site vitrine
SITE_DEPLOY_HOOK	l'URL du Deploy Hook Cloudflare du site
CLOUDINARY_*	à garder si les images restent sur Cloudinary ; à vider pour les stocker sur le disque du serveur

Mises à jour suivantes

Une mise à jour de **code** (pas de contenu : le contenu se fait dans l'admin) :

```
cd ~/optinov-admin
source ~/nodeenv/optinov-admin/22/bin/activate
git pull
NODE_ENV=development npm ci --include=dev
npm run migrate
NODE_OPTIONS="--no-deprecation --max-old-space-size=2048" \
NEXT_CPUS=1 RAYON_NUM_THREADS=1 TOKIO_WORKER_THREADS=1 UV_THREADPOOL_SIZE=1 \
  npx next build --webpack
mv node_modules ~/nodeenv/optinov-admin/22/lib/node_modules && ln -s ~/nodeenv/optinov-
admin/22/lib/node_modules node_modules
touch tmp/restart.txt
```

Le site immobilier a regroupé ces gestes dans un script `deployer.sh` : il peut servir de modèle.

Si ça ne démarre pas

1. Le journal de Passenger est dans le panneau (application → « Journal »), ou dans `~/logs/`. La première ligne d'erreur dit presque toujours de quoi il s'agit.
2. `node_modules` n'est plus un lien ? Refaire l'étape 6.
3. « Vous n'êtes pas autorisé à effectuer cette action » à l'enregistrement, alors que la consultation marche : `PAYLOAD_PUBLIC_SERVER_URL` ne correspond pas à l'adresse ouverte dans le navigateur.
4. Une variable d'environnement modifiée ne prend effet qu'après `touch tmp/restart.txt`.

10. Ce qui reste à faire après les fusions

Une fois les sept PR fusionnées et Render redéployé, ces gestes terminent la mise en service. Ils ne demandent pas de code, mais ils demandent les accès de l'agence.

A. Brancher les alertes e-mail (dès la fusion de #6)

1. Dans le **compte Google de l'agence** : Sécurité > Validation en deux étapes (l'activer si besoin) > **Mots de passe des applications** > créer un mot de passe nommé « Tableau de bord ». Google affiche seize caractères : les copier.
2. Dans **Render** > service `optinov-dashboard` > Environment, renseigner :

Variable	Valeur
<code>SMTP_USER</code>	l'adresse Gmail de l'agence
<code>SMTP_PASS</code>	les seize caractères du mot de passe d'application (jamais le mot de passe du compte)
<code>ALERTE_EMAIL</code>	l'adresse qui doit recevoir les alertes (peut être la même)

`SMTP_HOST` et `SMTP_PORT` sont déjà fixés dans `render.yaml`. Render redémarre le service tout seul.

1. **Tester avec un vrai formulaire** depuis le site en ligne (page Contact) : la demande doit apparaître dans le tableau de bord, rubrique Demandes ; l'alerte doit arriver sur `ALERTE_EMAIL` ; l'accusé de réception sur l'adresse saisie dans le formulaire. Ensuite, supprimer la demande de test dans le tableau de bord.
2. Si l'alerte n'arrive pas : Render > Logs, chercher « Alerte NON envoyée ». La cause est presque toujours un mot de passe d'application mal copié ou la validation en deux étapes désactivée.

B. Vérifier la reconstruction automatique du site

1. Dans **Cloudflare** > Workers & Pages > `optinov-agence` > Settings > Builds > **Deploy hooks** : créer un hook s'il n'existe pas, copier son URL.
2. Dans **Render** > Environment : `SITE_DEPLOY_HOOK` = cette URL.
3. Test de bout en bout : modifier un témoignage dans le tableau de bord, enregistrer, attendre deux minutes (regroupement) puis le temps du build Cloudflare (deux à trois minutes). La modification doit apparaître sur le site. Si rien ne bouge : Render > Logs, chercher « Reconstruction du site » ; puis Cloudflare > Deployments, vérifier qu'un build est parti et lire son journal.

C. Créer les comptes de l'équipe

Dans le tableau de bord > Administration > Utilisateurs > Ajouter : nom, e-mail, mot de passe provisoire, rôle. **Administrateur** pour les personnes qui gèrent les comptes, **Éditeur** pour celles qui ne touchent qu'au contenu. Chaque personne change ensuite son mot de passe depuis « Mon compte ». Supprimer les comptes de test qui ne servent plus.

D. Contenus à obtenir de la direction

Ces éléments sont encore écrits « [À compléter] » dans le code du site, ou vides. Ils ne se saisissent pas dans le tableau de bord : ils demandent une modification du code du site (une PR) une fois les valeurs arrêtées.

Élément	Où dans le code du site
Raison sociale, forme juridique, capital, coordonnées de l'hébergeur	<code>app/(site)/mentions-legales/page.js</code>
Date de mise à jour, référent protection des données, durées de conservation	<code>app/(site)/politique-de-confidentialite/page.js</code>
Prix des trois offres PROS.CARDS	<code>content/prosCards.js</code>
Chiffres clés et logos clients de l'accueil	<code>content/site.js</code> (<code>chiffresCles</code> , <code>logosClients</code>)

En revanche, **coordonnées, horaires, RCCM, directeur de publication, hébergeur, réseaux sociaux et liens PROS.CARDS** se saisissent directement dans le tableau de bord, rubrique Paramètres du site, et arrivent sur le site à la reconstruction suivante.

E. Remplir le tableau de bord

Le site affiche ce que contient le tableau de bord. À saisir pour que les pages correspondantes s'affichent : au moins un article de blog, les illustrations des cinq pages Services, les témoignages, l'équipe, les réalisations (cochées « Publiée »). Tant qu'une rubrique est vide, la section du site correspondante se masque ou affiche un message d'attente.

Aide-mémoire

Je veux...	Commande ou action
voir les PR ouvertes	onglet « Pull requests » du dépôt
tester une PR	<code>git fetch origin && git checkout <branche></code> puis <code>npm install</code>
fusionner	« Create a merge commit », dans l'ordre des numéros
proposer un changement	branche + commits + <code>git push -u origin <branche></code> + PR
compléter une PR ouverte	commits sur la même branche + <code>git push</code>
revenir à la version en ligne	<code>git checkout main && git pull</code>
ce qui a changé en production	historique de <code>main</code> sur GitHub, ou <code>git log --oneline main</code>
reconstruire le site	enregistrer n'importe quoi dans le tableau de bord, ou « Retry deployment » dans Cloudflare
redémarrer sur PlanetHoster	<code>touch tmp/restart.txt</code> dans le dossier de l'application